Recipe from your fridge

Kota OKUDA
Ryunosuke TANAKA

What we made

A database application that suggests dishes you can cook from the ingredients in your fridge

- Inputting the ingredients from your fridge, you can get the list of dishes you can cook
- With a filtering search (genre, difficulty...) (extensional feature)
- It also suggests dishes you can “almost” cook (extensional feature)

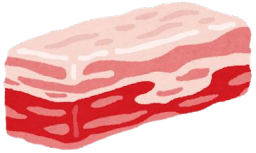
Database Design (Entities)

Table1: Dish

- Examples: Nikujaga (meat and potato stew), curry rice, meat sauce
- Attributes:
    - dish_id: unique internal identifier (primary key)
    - name: display name of the dish

Table2 Ingredient:

- Examples: carrot, onion, pork
- Attributes:
    - ingredient_id:
    - name: display name of the ingredient



Database Design (Relationship)

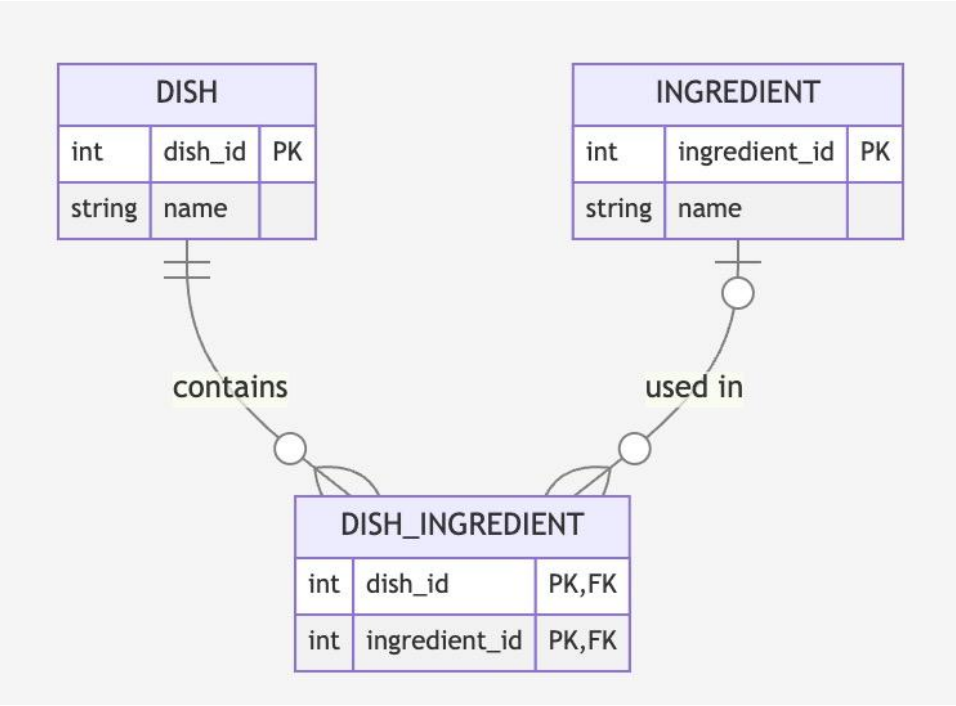
- Many-to-many relationship:
  - One dish contains many ingredients (nikujaga contains carrot, onion, pork, potato, …).
  - One ingredient is used in many dishes (carrot appears in nikujaga, in curry rice, …).

- Some participation constraints:

  - Dish side: total participation — a "dish with zero ingredients" has no meaning here, so every Dish must be connected to at least one Ingredient via `contains`.

  - Ingredient side: partial participation — an ingredient can exist in the database without being `used_in` any dish yet (e.g., *carrot* has been registered but no recipe uses it).

Database Design (ER Diagram & Relational Schema)



ER element	→	Relation
`Dish` entity	→	`Dish(dish_id, name)`
`Ingredient` entity	→	`Ingredient(ingredient_id, name)`
`contains` relationship (N:M)	→	`Dish_Ingredient(dish_id, ingredient_id)`

Database Design (Relational Schema)

- ‘Dish’ entity → Dish(dish_id, name)
- ‘Ingredient’ entity → Ingredient(ingredient_id, name)
- ‘contains’ relationship → Dish_Ingredient(dish_id, ingredient_id)

- The junction table `Dish_Ingredient` carries:
  - Composite primary key `(dish_id, ingredient_id)` — no duplicate (dish, ingredient) pairs allowed
  - Foreign key `dish_id` references `Dish.dish_id`
  - Foreign key `ingredient_id` references `Ingredient.ingredient_id`

Basic features

- 2.1 Find missing ingredients
  - used set difference

- 2.2 Partial Match of ingredients to dishes
  - The user has specific ingredients (e.g., Carrot, Celery) and wants to find all dishes in the database that use at least one of these items
  - “OR” search

- 2.3 Complete Match of ingredients to dishes
  - The user has a set of ingredients and wants to find dishes that can be prepared immediately without purchasing any additional items
  - “AND” search

Problems so far

- Staple Logic: "salt" or "oil" must be manually entered into the my fridge list for every query
- Substitution Logic: unable to automatically recognise "shallots" as a valid substitute when "onions" are missing
- Filtering Logic: If the search yields a large volume of results, it is better to narrow down recipes by genre or difficulty level

Extensional features / scenarios:

- Scenario 1: Dish using substitution
  - The user wants to make a Crepe, but they are missing Flour. However, they have Buckwheat Flour (in the fridge?)

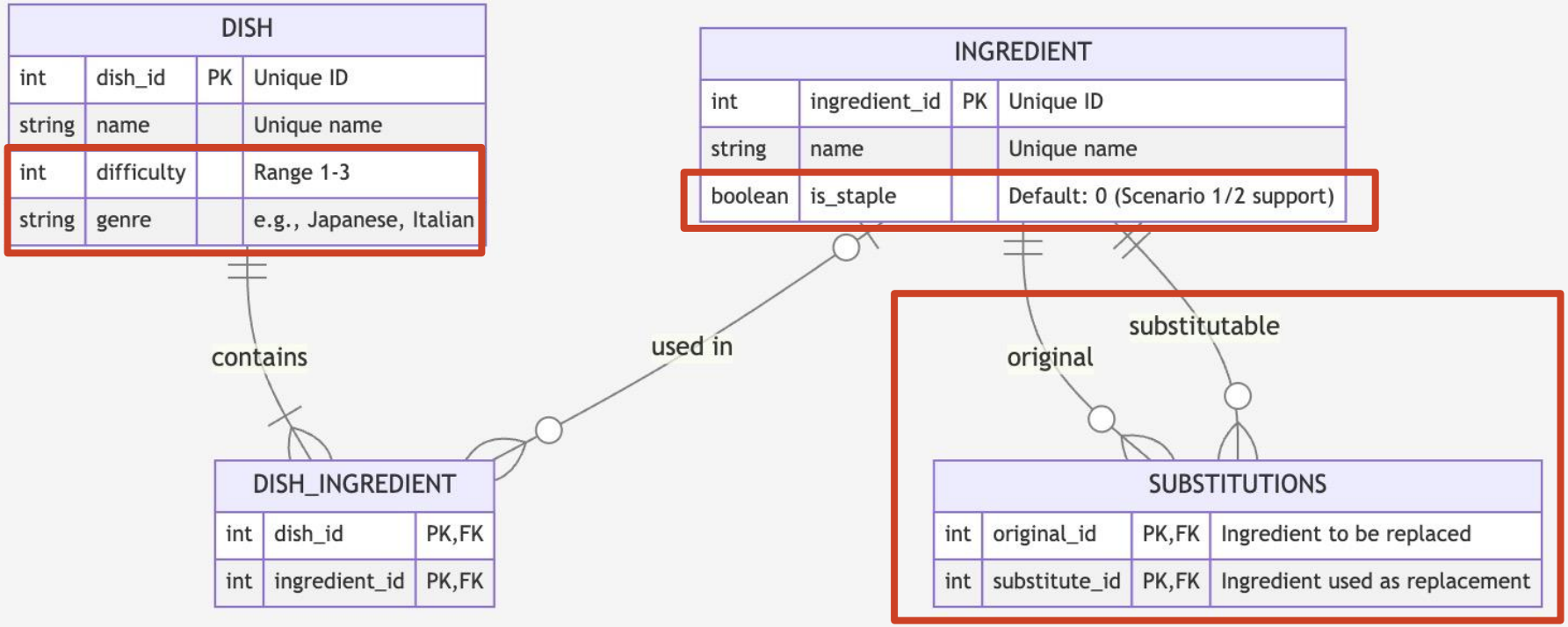
- Scenario 2: Multi-Criteria Filtering
  - The user wants to cook something specific: a Japanese dish that is easy to make (Difficulty ≤1)
  - They want to ensure they have all necessary ingredients (Complete Match) before starting

- Scenario 3: Near-Match Search
  - The user wants to find dishes they are "almost" able to cook.They are willing to go to the store, but only if they only need to buy exactly one more item

- Scenario 4: LLM-generated recipes
  - The user wants to cook dishes that are not in the database (for more flexibility)
  - A LLM generates a recipe from the ingredients in the fridge
  - The new recipe and ingredients are integrated to the database

New ER & Schema

ER element	→	Relation
`Dish` entity	→	`Dish(dish_id, name, difficulty, genre)`
`Ingredient` entity	→	`Ingredient(ingredient_id, name, is_staple)`
`contains` / `used_in` relationship (N:M)	→	`Dish_Ingredient(dish_id, ingredient_id)`
`Substitutable` relationship (N:M)	→	`Substitutions(original_id, substitute_id)`

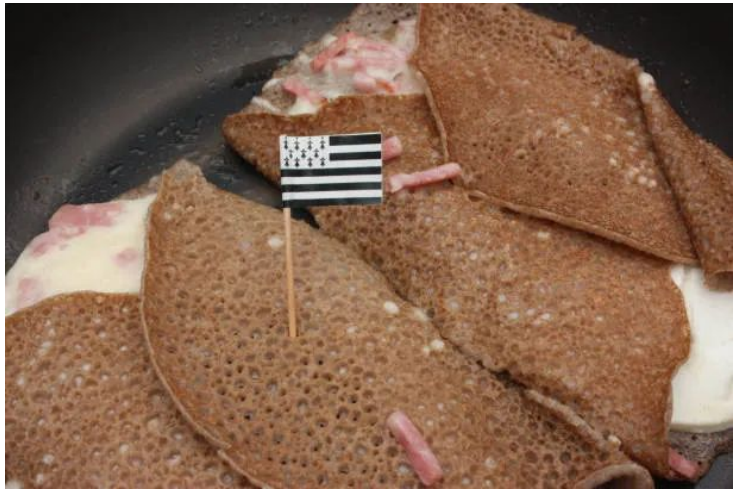


Scenario 1 (Dish using substitution)

Crêpe requires Flour, Egg, Milk

But what if we only have Buckwheat Flour instead of Flour?

→Galettes (Bretonnes)



Scenario 1 (Dish using substitution)

- R_d: Set of required ingredients for dish d
- A: Set of ingredients directly available in the fridge.
- Staple: Set of staples (Salt, Oil, etc.).
- S: Set of "virtual" ingredients covered by substitutes

  $S = \{i_{orig} \mid \exists i_{sub} \in A, (i_{orig}, i_{sub}) \in \text{Substitutions}\}.$
- Condition: The dish is selected if its requirements are met by the combination of direct ingredients, substitutes, and staples.

$$R_d \subseteq (A \cup \text{Staple} \cup S)$$

Scenario 2 (Multi-Criteria Filtering)

- R_d: Set of required ingredients for dish d.
- A: Set of available ingredients in the fridge.
- Staple: Set of staples (Salt, Oil, etc.).
- G: Target Genre ('Japanese').
- D: Max Difficulty level (from 1 to 3).

Condition: The dish is selected if it satisfies

$$R_d \subseteq (A \cup \text{Staple}) \wedge \text{genre}(d) = G \wedge \text{difficulty}(d) \leq D$$

Scenario 3 (Near-Match Search)

- R_d: Set of required ingredients for dish d.
- $A \cup \text{Staple}$
- M_d: The set of missing ingredients for a dish, calculated as the set difference: $M_d = R_d \setminus (A \cup \text{Staple})$.

Condition: A dish is selected if the size (cardinality) of the missing set is exactly 1.

$$|M_d| = |R_d \setminus (A \cup \text{Staple})| = 1$$

Scenario 4 (LLM-generated recipes)

- The user wants to cook a dish that is not in the dataset
- For the flexibility, we employed Claude API
- The generated recipe and the new ingredients concerned get registered to the database
- They are registered atomically:
  - All three tables updated together, or not at all
  - Keeps the database consistent under failure
  - Prevents orphan ingredients if any INSERT fails

Possible extensions

There still exists problems:

- Concurrent access
  - Force users to create accounts and identifies if there are two or logged-in from different devices and two or more queries with the same ingredient in a certain time (e.g. within 30 minutes)
  - Ingredients in `Staple` can be shared

- Ambiguous Recipe
  - What is tomato pasta? (a class of noodles, sauce/fresh/dry/whole tomatoes, cheese,)
  - `type` as a ingredient might be implemented;
    - pairwise sustitutable within a class of cheese?
    - Another problem: parmigiano and pecorino? parmigiano and mascarpone? smaller class of cheese

- Manual registeration
  - The user has to input the ingredients in their fridge manually everytime

We will switch to the demo